tut_threads/mutex_4.rb

```
rate_mutex = Thread::Mutex.new
exchange_rates = ExchangeRates.new
exchange_rates.update_from_online_feed

Thread.new do
  rate_mutex.lock
  loop do
    rate_mutex.sleep(3600)
    exchange_rates.update_from_online_feed
  end
end

loop do
  print "Enter currency code and amount: "
  line = gets
  if rate_mutex.try_lock
    begin
      puts(exchange_rates.convert(line))
    ensure
      puts "Ensuring unlock"
      rate_mutex.unlock
    end
  else
    puts "Sorry, rates being updated. Try again in a minute"
  end
end
```

A condition variable is a controlled way of communicating an event (or a condition) between two threads. One thread can wait on the condition, and the other can signal it. The Thread library extends threads with condition variables. See the Monitor library for an example.

Running Multiple External Processes

Sometimes you may want to split a task into several process-sized chunks—maybe to take advantage of all those cores in your shiny new processor. Or perhaps you need to run a separate process that was not written in Ruby. Not a problem: Ruby has a number of methods by which you may spawn and manage separate processes.

Spawning New Processes

You have several ways to spawn a separate process; the easiest is to run some command and wait for it to complete. You may find yourself doing this to run a system command or retrieve data from the host system. Ruby lets you spawn a process with the system or by using backquote (or backtick) methods:

```
system("tar xzf test.tgz") # => true
spawn("date")              # => 41176\nSun Dec 25 12:30:21 CST 2022
`date`                     # => "Sun Dec 25 12:30:21 CST 2022\n"
```

The method Object.system executes the given command in a subprocess; it returns true if the command was found and executed properly. It raises an exception if the command cannot be found. It returns false if the command ran but returned an error. In case of an error, you'll find the subprocess's exit code in the global variable $?. The spawn method is the same as